

第 7 章 PCIe 总线的数据链路层与物理层

PCIe 总线的数据链路层处于事务层和物理层之间，主要功能是保证来自事务层的 TLP 在 PCIe 链路中的正确传递，为此数据链路层定义了一系列数据链路层报文，即 DLLP。数据链路层使用了容错和重传机制保证数据传送的完整性与一致性，此外数据链路层还需要对 PCIe 链路进行管理与监控。数据链路层将从物理层中获得报文，并将其传递给事务层；同时接收事务层的报文，并将其转发到物理层。

与事务层不同，数据链路层主要处理端到端的数据传送。在事务层中，源设备与目标设备间的传送距离较长，设备之间可能经过若干级 Switch；而在数据链路层中，源设备与目标设备在一条 PCIe 链路的两端。因此本章在描述数据链路层时，将使用发送端与接收端的概念，而不再使用源设备与目标设备。

物理层是 PCIe 总线的最底层，也是 PCIe 总线体系结构的核心。在物理层中涉及许多与差分信号传递有关的模拟电路知识。PCIe 总线的物理层由逻辑层和电气层组成，其中电气层更为重要。在 PCIe 总线的物理层中，使用 LTSSM 状态机维护 PCIe 链路的正常运转，该状态机的迁移过程较为复杂，在第 8 章将详细介绍该状态机。

7.1 数据链路层的组成结构

数据链路层使用 ACK/NAK 协议发送和接收 TLP，由发送部件和接收部件组成。其中发送部件由 Replay Buffer、ACK/NAK DLLP 接收逻辑和 TLP 发送逻辑组成；而接收部件由“Error Check”逻辑、ACK/NAK 发送逻辑和 TLP 接收逻辑组成。数据链路层的拓扑结构如图 7-1 所示。在该图中含有两个 PCIe 设备，分别为 Device A 和 Device B，使用的 PCIe 链路为 Device A 的发送链路，同时也为 Device B 的接收链路。

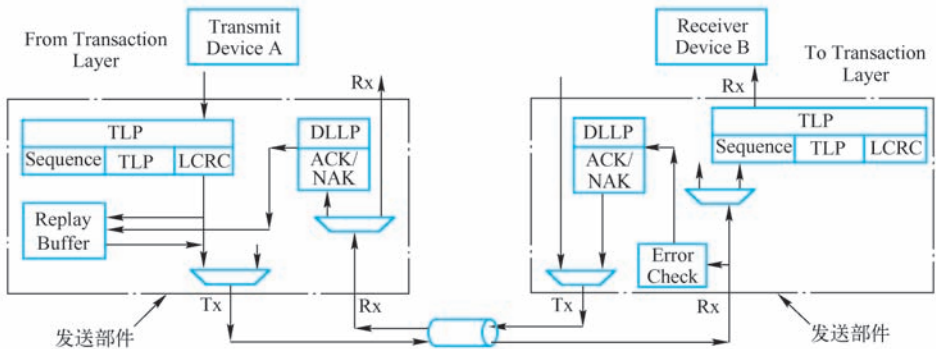


图 7-1 数据链路层的拓扑结构

实际上每个 PCIe 设备的数据链路层都含有发送部件和接收部件。而上图为简化起见，仅含有 Device A 的发送部件和 Device B 的接收部件，即 Device A 发送链路两端使用的两个

部件。Device A 和 Device B 也具有接收部件和发送部件，这两个部件由 Device B 的发送链路使用，Device B 发送链路的工作原理与 Device A 类似，本节对此不做详细介绍。

当 PCIe 设备进行数据传递时，首先在事务层中产生 TLP，然后通过事务层将这个 TLP 发送给数据链路层，数据链路层将这个 TLP 加上 Sequence 前缀和 LCRC 后缀后，首先将这个 TLP 放入到 Replay Buffer 中，然后再发送到物理层。

目标设备（Device B）从物理层接收 TLP 时，将首先获得带前后缀的 TLP，该 TLP 经过数据链路层传递给事务层时，将被去掉 Sequence 前缀和 LCRC 后缀。在数据链路层中，TLP 的格式如图 7-2 所示。

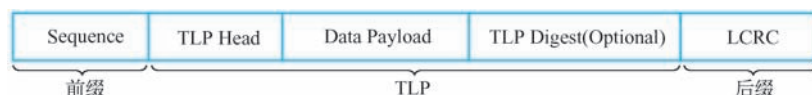


图 7-2 数据链路层 TLP 的格式

数据链路层使用 ACK/NAK 协议保证 TLP 的正确传送，ACK/NAK 协议是一种滑动窗口协议，该协议的详细介绍见第 7.2 节。其中 Sequence 前缀存放当前 TLP 的序列号，滑动窗口协议需要使用这个序列号。该序列号可以循环使用，但在同一个时间段内，一条 PCIe 链路不能含有 Sequence 前缀相同的多个 TLP。而 LCRC 后缀存放当前 TLP 的校验和。

PCIe 总线的数据链路层使用 Replay Buffer^①和 Error Check 部件共同保证数据传送的可靠性和完整性。来自事务层的 TLP 首先暂存在 Replay Buffer 中，然后发送到目标设备。源设备的数据链路层根据来自目标设备的 ACK/NAK DLLP 报文决定是重发这些 TLP，还是清除保存在 Replay Buffer 中的 TLP。

Replay Buffer 的大小决定了事务层可以暂存在数据链路层的报文数，Replay Buffer 的容量越大，在 PCIe 设备发送流水线中容纳的报文越多，从而也容易保证流水线不会因为发送部件出现 underrun 而中断，但是 Replay Buffer 的容量越大，占用的系统资源也越多，从而影响 PCIe 设备的功耗。在一个实际应用中，芯片设计者需要根据 PCIe 链路的延时确定数据链路层 Replay Buffer 的大小，在第 12.4.1 节中将进一步介绍 Replay Buffer 的大小与 PCIe 链路延时的关系。

在 PCIe 设备的数据链路层中，还含有一个 Error Check 单元。PCIe 设备使用 Error Check 单元检查接收到的 TLP，并决定如何向对端设备进行报文回应。如果 TLP 被正确接收，PCIe 设备将向对端设备发送 ACK DLLP^②；如果 TLP 没有被正确接收，PCIe 设备将向对端设备发送 NAK DLLP。

除了 ACK/NAK DLLP 之外，数据链路层还定义了一系列数据链路层报文 DLLP，以保证 PCIe 链路的正常工作。这些 DLLP 都产生于数据链路层，并终止于数据链路层，并不会传送到事务层。有关 DLLP 格式的详细描述见第 7.1.3 节。

7.1.1 数据链路层的状态

数据链路层需要通过物理层监控 PCIe 链路的状态，并维护数据链路层的“控制与管理

① PCIe 规范将这个 Replay Buffer 称为 Retry Buffer。

② 数据链路层为提高 PCIe 链路的利用率，并不会每成功接收一个 TLP 后，都发送一个 ACK DLLP。

状态机” (Data Link Control and Management State Machine, DLCMSM)。DLCMSM 状态机可以从物理层获得以下与当前 PCIe 链路相关的状态。

- DL_Inactive 状态。物理层通知数据链路层当前 PCIe 链路不可用。在当前 PCIe 链路的对端没有连接任何 PCIe 设备，或者没有检测到对端设备的存在时，数据链路层处于该状态。
- DL_Init 状态。物理层通知数据链路层当前 PCIe 链路可用，且物理层正处于链路初始化状态，此时数据链路层不能接收或者发送 TLP 和 DLLP。此时 PCIe 链路首先需要初始化 VC0 的流量控制机制，然后再对其他虚通路进行流量控制的初始化。有关流量控制的详细描述见第 9 章。
- DL_Active 状态。当前 PCIe 链路处于正常工作模式。此时物理层已完成 PCIe 链路训练或者重训练。有关链路训练的详细描述见第 8 章。

DLCMSM 状态机的迁移模型如图 7-3 所示。

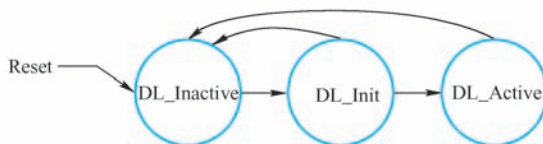


图 7-3 DLCMSM 状态机的迁移模型

DLCMSM 状态机除了可以使用上述状态位，从物理层获得当前 PCIe 链路状态外，还可以使用以下状态位，向事务层通知数据链路层所处的状态。事务层通过这些状态位获知数据链路层所处的工作状态。

- DL_Down。数据链路层处于该状态时，表示在 PCIe 链路的对端没有发现其他设备。当数据链路层处于 DL_Inactive 状态时，该状态位有效。值得注意的是 DL_Down 有效时，并不意味着对端不存在物理设备。数据链路层仅是使用该状态位通知事务层，暂时没有从对端中发现 PCIe 设备，需要进一步检测。
- DL_Up。数据链路层处于该状态表示在 PCIe 链路的对端连接了其他设备。当数据链路层处于 DL_Active 状态时，该状态位有效。

当数据链路层收到物理层的状态信息后，DLCMSM 状态机将进行状态转换，并向事务层通知 PCIe 链路的的状态。如果在 PCIe 链路的两端都连接着 PCIe 设备，那么这两个 PCIe 设备的数据链路层，在绝大多数时间内状态相同。数据链路层各个状态的详细说明，及 PCIe 链路的状态迁移过程如下。

1. DL_Inactive 状态

当 PCIe 设备复位时，将进入该状态。值得注意的是，只有传统复位方式才能使 PCIe 设备进入 DL_Inactive 状态，而 FLR 方式并不会影响 DLCMSM 状态机。

当 PCIe 设备从复位状态进入 DL_Inactive 状态时，将对 PCIe 数据链路层进行彻底复位，将与 PCIe 链路相关的寄存器置为复位值，并丢弃在 Replay Buffer 中保存的所有报文。当 PCIe 设备处于 DL_Inactive 状态时，数据链路层将向事务层提交 DL_Down 状态信息，并丢弃来自数据链路层和物理层的所有 TLP，而且不接收对端设备发送的 DLLP。

PCIe 设备的物理层设置了一个 LinkUp 位，该位为 1 时表示 PCIe 链路的对端与一个

PCIe 设备相连。当物理层的 LinkUp 状态位为 1，而且事务层没有禁用当前 PCIe 链路时，PCIe 数据链路层将从 DL_Inactive 状态迁移到 DL_Init 状态。

PCIe 设备在进行链路训练时，将检查 PCIe 链路的对端是否存在 PCIe 设备，如果对端不存在 PCIe 设备，物理层的 LinkUp 位将为 0，此时数据链路层将一直处于 DL_Inactive 状态。系统软件可以设置 Switch 下游端口 Link Control 寄存器的“Link Disable”位为 1，禁用该端口连接的 PCIe 链路，此时即便 PCIe 链路对端存在 PCIe 设备，数据链路层的状态也仍然为 DL_Inactive。

2. DL_Init 状态

当数据链路层处于 DL_Init 状态时，将对 PCIe 链路的虚通道 VC0 进行流量控制初始化。在 PCIe 总线中，流量控制的初始化分为两个阶段，分别为 FC_INIT1 和 FC_INIT2。在流量控制的 FC_INIT1 阶段，数据链路层将向事务层提交 DL_Down 状态信息；而在流量控制的 FC_INIT2 阶段，数据链路层将向事务层提交 DL_Up 状态信息。流量控制的初始化部分详见第 9.3.3 节。

当 PCIe 链路处于 DL_Down 状态时，发送端可以丢弃任何没有被 ACK/NAK 确认的 TLP，此时数据链路层几乎不会受到事务层的干扰，从而可以保证流量控制初始化的正常进行。这也是 PCIe 链路的流量控制分为 FC_INIT1 和 FC_INIT2 的主要原因。

当 VC0 的流量控制初始化完毕，而且物理层的 LinkUp 状态位为 0b1 时，数据链路层将从 DL_Init 状态迁移到 DL_Active 状态；如果在进行流量控制初始化时，物理层的 LinkUp 状态位被更改为 0b0 时，数据链路层将从 DL_Init 状态迁移到 DL_Inactive 状态。

3. DL_Active 状态

当数据链路层处于 DL_Active 状态时，PCIe 链路可以正常工作，此时数据链路层可以从事务层和物理层正常接收和发送 TLP，并处理 DLLP，此时数据链路层向事务层提交 DL_Up 状态信息。

当发生以下事件后，数据链路层可以从 DL_Active 状态迁移到 DL_Inactive 状态，但是不能迁移到 DL_Init 状态。这也意味着数据链路层从 DL_Active 状态迁移出去后，必须重新进行对端设备的识别和流量控制初始化，之后才能进入 DL_Active 状态。

在多数情况下，数据链路层从 DL_Active 状态迁移到 DL_Inactive 状态时，意味着处理器系统出现了异常，系统软件需要处理这些异常。但是在下列情况时，数据链路层状态从 DL_Active 状态迁移到 DL_Inactive 状态时并不会引发异常。

- Bridge Control Register 的 Secondary Bus Reset 位被系统软件置为 1 时，数据链路层将迁移到 DL_Inactive 状态。
- Link Disable 位被系统软件置为 1 时，数据链路层迁移到 DL_Inactive 状态。
- 当一个 PCIe 端口向对端设备发送“PME_Turn_Off”消息之后，其数据链路层经过一段时间，可以迁移到 DL_Inactive 状态。RC 和 Switch 在进入低功耗状态之前，将向其下游端口广播 PME_Turn_Off 消息，下游 PCIe 设备收到该消息后，将向 RC 和 Switch 发出 PME_TO_Ack 回应。当 RC 和 Switch 的下游端口收到这个回应报文后，数据链路层可以迁移到 DL_Inactive 状态。
- 如果 PCIe 链路连接了一个支持“热插拔”功能的 PCIe 插槽，而当这个插槽的 Slot Capability 寄存器的“Hot Plug Surprise”位为 1 时，数据链路层将迁移到 DL_Inactive

状态。

- 如果 PCIe 链路连接一个热插拔插槽，当这个插槽的 Slot Control 寄存器的“Power Controller Control”位为 1 时，数据链路层也将迁移到 DL_Inactive 状态。

在 PCIe 总线中，还有一些系统事件也可以引发数据链路层的状态转换，本书对此不进行一一描述。

7.1.2 事务层如何处理 DL_Down 和 DL_Up 状态

当事务层收到数据链路层的 DL_Down 状态信息时，表示出现了以下情况。

- PCIe 链路的对端没有连接设备。
- PCIe 链路丢失了与对端设备的连接。
- 数据链路层和物理层出现某种错误，PCIe 链路不能正常工作。
- 系统软件禁用 PCIe 链路。

当事务层收到 DL_Down 状态信息后，将不再从数据链路层中接收 TLP，除非是数据链路层已经使用 ACK/NAK 报文确认过的 TLP。这些被确认过的 TLP 已经被数据链路层接收完毕，因此事务层可以接收这些 TLP。

当链路处于 DL_Down 状态时，RC 或者 Switch 的下游端口将复位与链路相关的内部逻辑和状态。此时下游端口收到上游端口的 Non-Posted 请求 TLP 后，并不会将这个 TLP 转发到数据链路层，因为数据链路层已经出现故障，而组织状态位为 UR（Unsupported Request）的完成报文，通知上游端口无法发送这个 Non-Posted 数据请求，该事务层将丢弃这个 Non-Posted 请求 TLP；此外该事务层还将丢弃来自上游端口的 Posted 请求 TLP 和完成报文。当链路为 DL_Down 状态时，RC 或者 Switch 的下游端口还必须结束“PME Turn_Off”握手请求。

当链路为 DL_Down 状态时，Switch 和 PCIe 桥的上游端口，将复位相关的内部逻辑和状态，并丢弃所有正在处理的 TLP。此时 Switch 和 PCIe 桥将使用 Hot Reset 方式复位所有下游端口。

事务层处于 DL_Up 状态时，表示该设备与 PCIe 链路的对端设备已经建立连接，链路两端可以正常收发报文。当事务层发现 PCIe 链路从 DL_Down 迁移到 DL_Up 状态时，将向 PCIe 链路的对端设备重新发送 Set_Slot_Power_Limit 消息，并重新初始化相关的寄存器。

7.1.3 DLLP 的格式

DLLP 与 TLP 的概念并不相同，DLLP 产生于数据链路层，终止于数据链路层，这些报文不会出现在事务层中，而且对系统软件透明。设置 DLLP 的目的是为了保证 TLP 的正确传送和管理 PCIe 链路。

值得注意的是，DLLP 并不是由 TLP 加上 Sequence 前缀和 LCRC 后缀组成的，而具有单独的格式。一个 DLLP 由 6 个字节组成，其中第 1 个字节存放 DLLP 的类型，第 2~4 个字节存放的数据与 DLLP 类型相关，而最后两个字节存放当前 DLLP 的 CRC 校验。DLLP 的格式如图 7-4 所示。

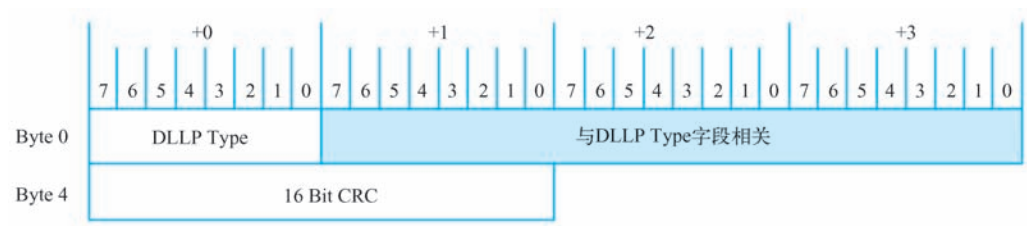


图 7-4 DLLP 的格式

大多数 DLLP 由 PCIe 设备自动产生，而与事务层没有直接联系。PCIe 总线定义了以下几类 DLLP 报文，如表 7-1 所示。

表 7-1 DLLP 的编码

Type 字段	DLLP 类型
0000-0000	ACK
0001-0000	NAK
0010-0000	PM_Enter_L1
0010-0001	PM_Enter_L23
0010-0011	PM_Active_State_Request_L1
0010-0100	PM_Request_Ack
0011-0000	Vendor Specific-Not used in normal operation
0100-0V ₂ V ₁ V ₀	InitFC1-P, 其中 V[2:0]由三位组成, 因为一条 PCIe 链路最多支持 8 个 VC
0101-0V ₂ V ₁ V ₀	InitFC1-NP
0110-0V ₂ V ₁ V ₀	InitFC1-Cpl
1100-0V ₂ V ₁ V ₀	InitFC2-P
1101-0V ₂ V ₁ V ₀	InitFC2-NP
1110-0V ₂ V ₁ V ₀	InitFC2-Cpl
1000-0V ₂ V ₁ V ₀	UpdateFC-P
1001-0V ₂ V ₁ V ₀	UpdateFC-NP
1010-0V ₂ V ₁ V ₀	UpdateFC-Cpl

这些 DLLP 报文的描述如下。

- ACK DLLP。该 DLLP 由接收端发向发送端。接收端收到 TLP 报文后，将根据数据链路层的阈值设置，向对端设备发送 ACK DLLP，而不是每接收到一个 TLP，都向对端发送一个 ACK DLLP。该 DLLP 表示接收端正确收到来自对端的 TLP。
- NAK DLLP。该 DLLP 由接收端发向发送端。该 DLLP 表示接收端有哪些 TLP 没有被正确接收，发送端收到 NAK DLLP 后，将重传没有被正确接收的 TLP，同时释放已经被正确接收的 TLP。ACK 和 NAK DLLP 与 ACK/NAK 协议相关，是数据链路层的两个重要 DLLP。这两个 DLLP 的详细作用如第 7.2 节所述。
- Power Management DLLPs。PCIe 设备使用该组 DLLP 进行电源管理，并向对端设备通知当前 PCIe 链路的状态。PCIe 总线还定义了一组与电源管理相关的 TLP，这些 TLP

与这组 DLLPs 有一定的联系，但是其作用并不相同。PCIe 总线使用该组 DLLP 保证电源管理状态机的正确运行。

- Flow Control Packet DLLPs。该组 DLLP 包括 InitFC1、InitFC2、UpdateFC DLLP，PCIe 总线使用这些 DLLPs 进行流量控制。在 PCIe 总线中，数据传送由三大类组成，分别为 Posted、Non-Posted 和 Completion。这三种数据传送方式有些细微区别，PCIe 设备为这三种数据传送设置了不同的数据缓冲。流量控制是 PCIe 总线的一个重要特性，第 9 章将重点介绍这些内容。
- Vendor-Specific DLLP。一些定制的 DLLP，PCIe 总线规范并未对此约束。这些 DLLP 由用户自定义使用。

本节将重点介绍 ACK/NAK DLLP，这两个 DLLP 与 PCIe 总线的 ACK/NAK 协议直接相关。在 PCIe 总线中，数据链路层使用 ACK/NAK 协议保证 TLP 的可靠传送。ACK/NAK DLLP 的格式如图 7-5 所示。

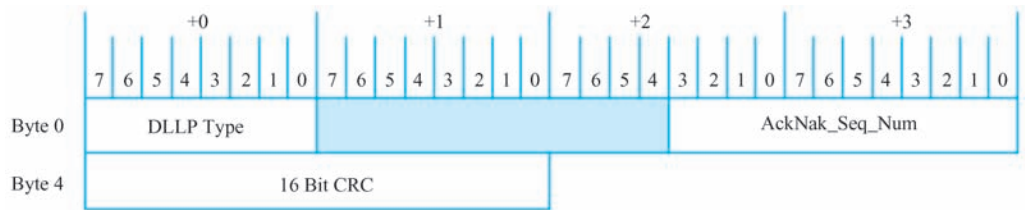


图 7-5 ACK/NAK DLLP 的格式

ACK/NAK DLLP 各字段的详细说明如表 7-2 所示。

表 7-2 ACK/NAK DLLP 的详细说明

名 称	位 置	描 述
DLLP Type	Byte 0 的 7~0 位	0b0000-0000 表示为 ACK 报文 0b0001-0000 表示为 NAK 报文
AckNak_Seq_Num	Byte 2 的 3~0 位， Byte 3 的 7~0 位	该字段表示接收端成功接收的报文序号，下文将详细解释该字段
CRC	Byte 4~Byte 5	保存 DLLP 的 CRC 校验和

发送端的数据链路层负责将 TLP 传送给接收端，而接收端的数据链路层在收到 TLP 之后，将向发送端发送 ACK/NAK DLLP。发送端和接收端通过某种传送协议，完成数据链路层的数据交换，在 PCIe 总线中，这个协议称为 ACK/NAK 协议。

7.2 ACK/NAK 协议

ACK/NAK 协议是一种滑动窗口协议。PCIe 设备的发送端和接收端分别设置了两个窗口。发送端在发送 TLP 时，首先将这个 TLP 放入发送窗口中（这个窗口即 Replay Buffer），并对这些 TLP 从 0~n 进行编号。只要发送窗口不满，发送端就可以持续地从事务层中接收报文，然后将其放入 Replay Buffer 中。

发送端需要保留在这个窗口中的数据报文，并在收到来自接收端的 ACK/NAK 确认报文之后，统一释放保存在发送窗口中的报文，并滑动这个发送窗口。当发送端收到接收端对第

n 个报文的确认后，表示第 n、n-1、n-2 等在窗口中的报文都已经被正确收到，然后统一滑动这个窗口。PCIe 总线使用这种方法可以提高窗口的利用率。

与此对应，接收端也维护了一个窗口，该窗口记录数据报文的发送序列号范围。当数据报文到达后，如果其序列号在接收窗口范围内，接收端将接收该报文，并根据实际情况，向发送端发送回应报文。这个回应报文包括 ACK 和 NAK DLLP。下文将分别讨论发送端和接收端如何使用 ACK/NAK 协议。

7.2.1 发送端如何使用 ACK/NAK 协议

数据链路层在发送 TLP 之前，发送端首先需要将 TLP 进行封装，加上 Sequence 前缀和 LCRC 后缀，之后再将这个 TLP 放入 Replay Buffer 中。发送端设置了一个 12 位的计数器 NEXT_TRANSMIT_SEQ，这个计数器的初始值为 0，当数据链路层处于 DL_Inactive 状态时，该计数器将保持为 0。为简化起见，本节只讲述数据链路层处于 DL_Active 状态时的情况，而不讲述处于 DL_Inactive 状态时的情况。

发送端使用计数器 NEXT_TRANSMIT_SEQ 的当前值设置 TLP 的 Sequence 号，该计数器的初始值为 0。PCIe 设备每发送完毕一个 TLP，这个计数器将加 1，直到该计数器的值为 4095 (NEXT_TRANSMIT_SEQ 的最大值)。当计数器的值为 4095 后，再进行加 1 操作时，该计数器将回归为 0。而 LCRC 是根据 TLP 的内容计算出来的，用来保证数据传递的完整性，本节不介绍 LCRC 的计算过程。对此有兴趣的读者请参考 PCIe 总线规范。

与此对应，接收端也设置了一个 12 位的计数器 NEXT_RCV_SEQ。这个计数器记录接收端即将接收的 TLP 的 Sequence 号。这个计数器的初始值为 0，当数据链路层处于 DL_Inactive 状态时，该计数器保持为 0。在正常情况下，到达接收端的 TLP，其 Sequence 号 and 这个计数器中的内容一致。当接收端将这个 TLP 转发到事务层后，这个计数器将加 1，当计数器的值为 4095 后，再进行加 1 操作时，该计数器将回归为 0。如果到达接收端的 TLP，其序号与这个计数器中的值不一致时，接收端需要进行特殊处理，详见下文。

发送端为处理来自接收端的 ACK/NAK DLLP，设置了一个 12 位的计数器 ACKD_SEQ。这个计数器记载最近接收到的 ACK/NAK DLLP 的 AckNak_Seq_Num 字段。这个计数器的初始值为全 1，当数据链路层处于 Inactive 状态时，该计数器保持为全 1。发送端收到 ACK/NAK DLLP 后，将使用这些 DLLP 中的 AckNak_Seq_Num 字段更新 ACKD_SEQ 计数器。

如果 $(\text{NEXT_TRANSMIT_SEQ} - \text{ACKD_SEQ}) \bmod 4096 \geq 2048$ 时，发送端将不会从事务层继续接收新的 TLP，因为此时发送端已经发送了许多 TLP，但是接收端可能并没有成功接收这些 TLP，因此并没有及时发送 ACK/NAK DLLP 作为回应。在多数情况下，当 PCIe 链路出现了某些问题时，才可能导致该公式成立。此外 ACKD_SEQ 计数器还可以帮助发送端重发错误的 TLP，下文将详细解释这个功能。

发送端首先将从事务层获得的 TLP 存放到 Replay Buffer 中，在 Replay Buffer 中可以存放多个 TLP，这个 Replay Buffer 为发送端使用的发送窗口。PCIe 总线并没有规定在 Replay Buffer 中存放 TLP 的个数，不同的设计可以采用的大小不同，其中有一个重要的原则就是不能使 Replay Buffer 成为整个设计的瓶颈，Replay Buffer 应该始终保证有足够的空间接收来自事务层的报文。

TLP 进入 Replay Buffer 之后，发送端首先将这个 TLP 封装，然后从 Replay Buffer 中发送到物理层，最终达到接收端。发送端将 TLP 发送出去之后，将等待来自接收端的应答，接收端使用 ACK/NAK DLLP 发送这个应答。发送端根据应答结果决定是将 TLP 从 Replay Buffer 中清除，还是重发在 Replay Buffer 中的 TLP。下文将以几个实例说明发送端如何处理来自接收端的应答。

1. 发送端收到 ACK DLLP 报文

如图 7-6 所示，假设发送端从 Replay Buffer 中向接收端发送 Sequence 号为 3~7 的报文。接收端收到这些报文后将发送 ACK DLLP 作为回应，其详细步骤如下。

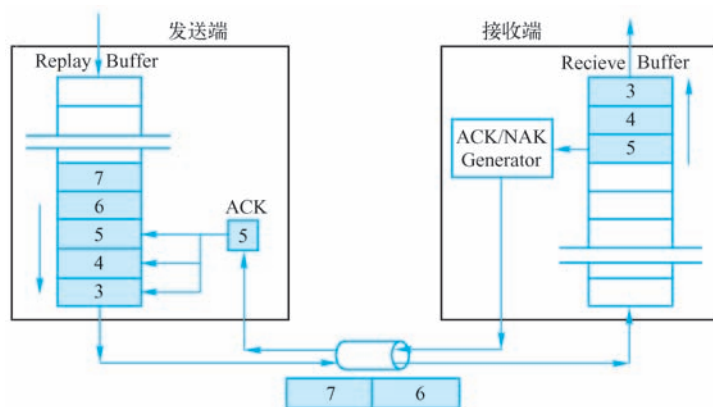


图 7-6 发送端收到 ACK DLLP 报文

1) 发送端向接收端发送 TLP3~7，其中 TLP3 是第一个报文，TLP7 是最后一个报文。此时发送端的 NEXT_TRANSMIT_SEQ 计数器为 8，表示即将填入到 Replay Buffer 中的报文序列号为 8。

2) 接收端按序收到 TLP3~5，而 TLP6 和 7 仍在传送过程中。接收端的 NEXT_RCV_SEQ 计数器为 6，表示即将接收的报文序列号为 6。

3) 接收端通过报文检查决定接收 TLP3~5，然后发送 ACK DLLP，此时这个 ACK DLLP 的 AckNak_Seq_Num 字段为 5。为了提高总线的利用率，接收端不会为每一个接收到的 TLP 都做出应答。在这个例子中，AckNak_Seq_Num 字段为 5 表示 TLP3~5 都被接收。

4) 发送端收到 AckNak_Seq_Num 字段为 5 的 ACK DLLP 后，得知 TLP3~5 都被成功接收。此时发送端将 TLP3~5 从 Replay Buffer 中清除。

5) 接收端陆续收到 TLP6~7 后，接收端的 NEXT_RCV_SEQ 计数器为 8，表示即将接收的报文序列号为 8。然后接收端向发送端发送 ACK DLLP，这个 DLLP 的 AckNak_Seq_Num 字段为 7，即为 NEXT_RCV_SEQ-1。

6) 发送端收到 AckNak_Seq_Num 字段为 7 的 ACK DLLP 后，得知 TLP6~7 都被成功接收。此时发送端将 TLP6~7 从 Replay Buffer 中清除。

2. 发送端收到 NAK DLLP 报文

如图 7-7 所示，假设发送端从 Replay Buffer 中向接收端发送 Sequence 号为 3~7 的报文。接收端收到这些报文后，发现有错误的 TLP，此时将发送 NAK DLLP 而不是 ACK DLLP，其

详细步骤如下。

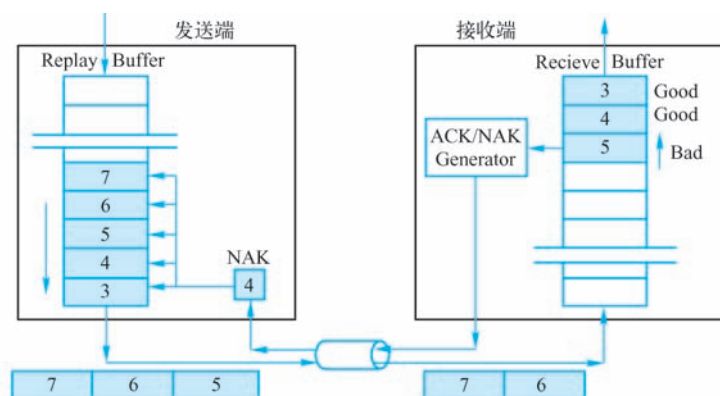


图 7-7 发送端收到 NAK DLLP 报文

- 1) 发送端向接收端发送 TLP3~7，其中 TLP3 是第一个报文，而 TLP7 是最后一个报文。
- 2) 接收端按序收到 TLP3~5，而 TLP6 和 7 仍在传送过程中。
- 3) 接收端通过报文检查决定接收 TLP3~4，此时 NEXT_RCV_SEQ 为 5，表示即将接收 TLP5。
- 4) TLP5 没有通过完整性验证，此时接收端将向对端发送 NAK DLLP，这个 DLLP 的 AckNak_Seq_Num 字段为 4，即为 NEXT_RCV_SEQ-1。AckNak_Seq_Num 字段为 4 表示接收端最后一个接收正确的 TLP，其 Sequence 号为 4。
- 5) 发送端收到 AckNak_Seq_Num 字段为 4 的 NAK DLLP 后，得知 TLP3~4 已被成功接收。此时发送端首先停止从事务层接收新的 TLP，之后将 TLP3~4 从 Replay Buffer 中清除。
- 6) 发送端重新发送在 Replay Buffer 中从 TLP5 开始的报文。在这个例子中，发送端将重新发送 TLP5~7。

发送端每一次收到 NAK DLLP 后，都将重发在 Replay Buffer 中剩余的 TLP。但是发送端不能无限次重发同一个 TLP，因为出现这种情况意味着链路出现了某些问题，必须修复这些问题后，才能继续重发这些 TLP。为此在发送端中设置了一个 2 位计数器 REPLAY_NUM，这个计数器的初始值为 0，当数据链路层处于 Inactive 状态时，该计数器保持为 0。

REPLAY_NUM 计数器按照以下几个原则进行更新。

- 当发送端第一次收到 NAK DLLP 后，REPLAY_NUM 计数器将加 1，此时 ACKD_SEQ 计数器被赋值为这个 NAK DLLP 的 AckNak_Seq_Num 字段。之后当发送端收到新的 ACK/NAK DLLP，而且其 AckNak_Seq_Num 字段大于 ACKD_SEQ 计数器的值时（表示发送端至少重传成功一个 TLP），REPLAY_NUM 计数器将被重置为 0，ACKD_SEQ 计数器的值也更新为相应的值。
- PCIe 总线规定发送端新收到的 ACK/NAK DLLP，其 AckNak_Seq_Num 字段不能小于 ACKD_SEQ 计数器，如果出现这种问题，将是芯片的 Bug。
- 如果新的 ACK/NAK DLLP，其 AckNak_Seq_Num 字段值等于 ACKD_SEQ 计数器的值时，表示发送端正在反复地重新发送同一个 TLP，此时 REPLAY_NUM 计数器将加 1，当这个计数器溢出时，发送端将不再重复发送这个 TLP，而是重新进行链路训练，当

PCIe 链路恢复正常后，再重新发送这个 TLP。

发送端除了设置 REPLAY_NUM 计数器，判断 PCIe 链路可能出现的故障之外，还设置了另一个计数器 REPLAY_TIMER，进一步识别 PCIe 链路可能出现的故障。因为使用 ACKD_SEQ 计数器判断链路故障的基础是发送端可以收到 ACK/NAK DLLP。但是在某种情况下，发送端虽然发送了 TLP，但是接收端没有回应 ACK/NAK DLLP，或者由于 PCIe 链路故障发送端没有收到这个回应。

此时发送端需要使用 REPLAY_TIMER 计数器，以判断在 TLP 的传送过程中是否出现异常。REPLAY_TIMER 计数器记载一个 TLP 报文从发送到获得 ACK/NAK DLLP 回应的时间，当这个时间过长时，发送端认为 PCIe 链路出现故障。REPLAY_TIMER 计数器的更新规则如下所示。

1) REPLAY_TIMER 计数器的初始值为 0，在发送端发出或者重新发出一个 TLP 后以一个固定的时钟频率开始计数，当 REPLAY_TIMER 计数器到达设定的阈值时，将认为 PCIe 链路出现故障，此时 PCIe 设备将进行 PCIe 链路的恢复工作。如果发送端可以正常收到 ACK/NAK DLLP 时，该计数器不会溢出。

2) 发送端收到 ACK DLLP，而且在 Replay Buffer 中没有“已经发送而且尚未被确认的 TLP”后，REPLAY_TIMER 计数器被重置并重新开始计数。在正常情况下，发送端每发送一个 TLP 后，REPLAY_TIMER 计数器将被重置并重新计数，但是有时在 Replay Buffer 中存在一些 TLP，这些 TLP 已经被发送出去，但是并没有收到相应的 ACK DLLP，此时 REPLAY_TIMER 计数器不能被重置。

3) 当 Replay Buffer 中没有任何 TLP 时，REPLAY_TIMER 计数器将被重置而且其值保持不变。

4) 发送端收到 NAK DLLP 之后，REPLAY_TIMER 计数器将被重置而且其值保持不变。当数据链路层重新发送 TLP 时，REPLAY_TIMER 计数器才重新开始计数。

5) REPLAY_TIMER 计数器到达设定的阈值后，该计数器的值将被重置，且保持不变。此时 PCIe 链路可能出现某种异常，当这些异常被处理完毕后，REPLAY_TIMER 计数器才可能重新计数。

6) PCIe 链路重新训练时，REPLAY_TIMER 计数器的值保持不变。准确地说，PCIe 链路处于 Recovery 或者 Configuration 状态时，REPLAY_TIMER 计数器的值保持不变。有关 Recovery 或者 Configuration 状态的详细说明见第 8.2 节。

PCIe 总线规范提供了计算 REPLAY_TIMER 计数器阈值的经验公式，这个阈值和 PCIe 设备和主桥提供的 Max_Payload_Size 成正比，与 PCIe 链路宽度成反比，本节对此公式不进行详细说明。值得注意的是，这个经验公式是基于 x86 处理器计算得出的，不同的处理器在此处的实现不尽相同。

7.2.2 接收端如何使用 ACK/NAK 协议

接收端首先从物理层获得 TLP，此时在这个 TLP 中包含 Sequence 号前缀和 LCRC 后缀。接收端收到这个 TLP 后，首先将这个报文放入 Receive Buffer 中，然后进行 CRC 检查。如果 CRC 检查成功，接收端将根据接收缓冲的阈值发送 ACK DLLP 给发送端，并将这个 TLP 传给事务层。除了 CRC 校验外，接收端还需要做其他检查，本节对此不进行介绍。

1. 接收端发送 ACK DLLP

当接收端收到的 TLP 没有出现 LCRC 错误，而且 TLP 的 Sequence 号和 NEXT_RCV_SEQ 计数器的值相同时，接收端将正确接收这个 TLP，并将其转发给事务层，随后接收端将 NEXT_RCV_SEQ 计数器加 1。

接收端根据具体情况，决定向发送端立即发送 ACK DLLP，还是等待接收到更多的 TLP 后再发送 ACK DLLP。如果接收端决定发送 ACK DLLP，则该 ACK DLLP 的 AckNak_Seq_Num 字段为 NEXT_RCV_SEQ-1，即已经正确接收 TLP 的 Sequence 号。

接收端不会对每一个正确接收的 TLP 发出 ACK DLLP 回应，因为这样将严重影响 PCIe 总线链路的使用效率，而是收集一定数量的 TLP 后，统一发出一个 ACK DLLP 回应表示之前的 TLP 都已正确接收。

为此接收端使用了一个 ACKNAK_LATENCY_TIMER 计数器，当这个计数器超时或者接收的报文数超过一个阈值后，向发送端发送一个 ACK DLLP 回应。此时这个 ACK DLLP 的 AckNak_Seq_Num 字段为在这段时间以来，最后一个被正确接收的 TLP 的 Sequence 号。不同的设计在此处的实现不尽相同。但是这些实现都要遵循以下两个原则。

1) 接收端在收到一定数量的报文后，统一发送一个 ACK DLLP 作为回应。

2) 接收端收到的报文虽然没有到达阈值，但是 ACKNAK_LATENCY_TIMER 计数器超时后，仍然要发出 ACK DLLP 作为回应。在某些情况下，发送端可能在发送一个 TLP 后，在很长一段时间内，都不会发送新的 TLP，此时接收端必须及时给出 ACK DLLP 回应，以免发送端的 REPLAY_TIMER 计数器溢出。

下面将以一个实例说明接收端如何发送 ACK DLLP 回应，该实例如图 7-8 所示，其描述如下所示。

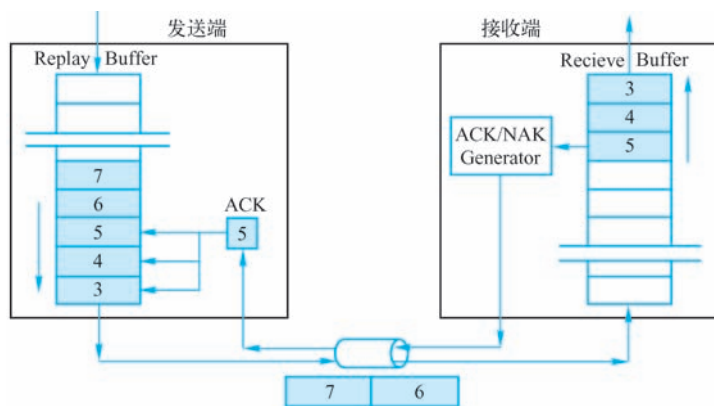


图 7-8 接收端发送 ACK DLLP

1) 发送端发送 TLP3~7 给接收端，其中 TLP3 是第一个报文，而 TLP7 是最后一个报文。

2) 接收端按序收到 TLP3~5，而 TLP6 和 7 仍在传送过程中。此时 NEXT_RCV_SEQ 的值被更新为 6，表示下一个即将接收的 TLP，其 Sequence 号为 6。

3) 接收端通过报文检查决定接收 TLP3~5，然后发送 ACK DLLP，这个 DLLP 的 AckNak_

Seq_Num 字段为 5。为了提高总线的利用率，接收端不会对每一个接收到的 TLP 都做出应答。在这个例子中，AckNak_Seq_Num 字段为 5 表示 TLP3~5 已经被接收。

4) 接收端将接收到的 TLP3~5 传递给事务层。

5) 接收端陆续收到 TLP6~7 后，继续执行步骤 3~4。

2. 接收端发送 NAK DLLP

接收端设置了一个 NAK_SCHEDULED 位，该位用来判断接收端如何发送 NAK DLLP，该位的初始值为 0。当接收端接收 TLP 出现错误时，将该位置为 1；当出现“错误的 TLP”被重新接收成功后，该位被置为 0。

如果接收端发现 TLP 的 CRC 错误后，将丢弃这个 TLP，并发送 NAK DLLP，这个 DLLP 的 AckNak_Seq_Num 字段为 NEXT_RCV_SEQ-1，即最后一个正确接收 TLP 的 Sequence 号。此时 NEXT_RCV_SEQ 计算器将保持不变，NAK_SCHEDULED 位将被置为 1。

当这个 NAK DLLP 到达发送端之前，PCIe 链路还有一些正在传送的 TLP，这些报文将陆续到达接收端，这些报文的 Sequence 号都将大于 NEXT_RCV_SEQ，此时接收端不会接收这些报文，而且接收端也不会为这些报文发送 NAK DLLP，因为此时 NAK_SCHEDULED 位继续保持有效，即为 1。

PCIe 总线规范没有规定接收端如何拒收后续的 TLP 报文，较为合理的实现方式是这些后续的报文无须进入接收端的 Receive Buffer 就被拒绝。这样接收端只为已经进入 Receive Buffer 中的 TLP 发出 ACK/NAK 回应。

如果接收端收到一个重试的 TLP 报文，其 Sequence 号与 NEXT_RCV_SEQ 相等，而且报文没有出现错误，接收端将 NEXT_RCV_SEQ 加 1 同时清除 NAK_SCHEDULED 位。此时表示重试的报文已经被正确接收。

如果接收端收到的 TLP，其 Sequence 号小于 NEXT_RCV_SEQ，这种情况通常是由于 TLP 传送过程中的延时，产生的重复 TLP，此时接收端将丢弃这个报文，NEXT_RCV_SEQ 计数器的值也不会改变，随后接收端将向对端发送 ACK DLLP，这个 DLLP 的 AckNak_Seq_Num 为 NEXT_RCV_SEQ-1。

[Ravi Budruk, Don Anderson and Tom Shanley]列举了一个实例说明在某种情况下，接收端将收到此类报文。接收端正确接收到 TLP 后，将发送 ACK DLLP，但是由于链路故障，这个报文并没有到达发送端，此时已经发送的 TLP 将不会从发送端的 Replay Buffer 中清除，最终 REPLAY_TIMER 将溢出，此时发送端有可能重新进行链路训练，当链路恢复正常后，发送端将重新发送 Replay Buffer 中的所有 TLP。在这种情况下，接收端将收到 Sequence 号比 NEXT_RCV_SEQ 小的 TLP。

下面将使用一个实例进一步说明接收端如何发送 NAK DLLP，该实例如图 7-9 所示，其描述如下所示。

1) 发送端向接收端发送 TLP3~7，其中 TLP3 是第一个报文，而 TLP7 是最后一个报文。

2) 接收端按序收到 TLP3~5，并将这些报文放入 Receive Buffer，当然也可以在这些报文通过完整性检查后，再决定是否将这些 TLP 放入 Receive Buffer 中，而 TLP6 和 7 仍在传送过程中。

3) 接收端通过报文检查决定接收 TLP3~4，此时 NEXT_RCV_SEQ 为 5，表示即将接收

TLP5。此时接收端将 TLP3~4 传递给事务层。

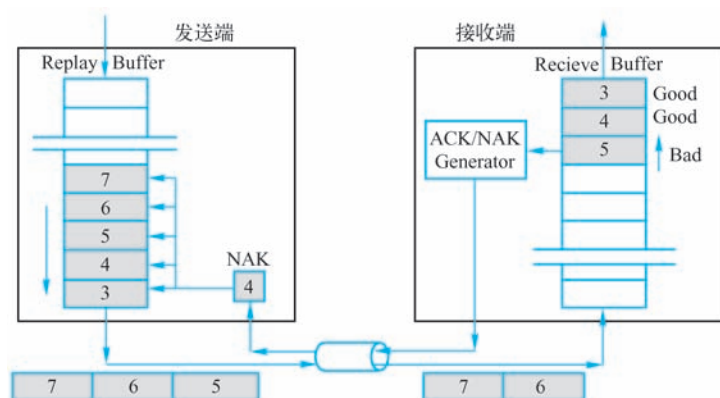


图 7-9 接收端如何发送 NAK DLLP

4) 而 TLP5 没有通过完整性验证, 此时接收端将发送 NAK DLLP, 这个 DLLP 的 AckNak_Seq_Num 字段为 4, 即 NEXT_RCV_SEQ-1。AckNak_Seq_Num 字段为 4 表示接收端最后一个正确接收的 TLP, 其 Sequence 号为 4。此时接收端将设置 NAK_SCHEDULED 位为 1, 而 NEXT_RCV_SEQ 保持不变, 即为 5。

5) 接收端将丢弃 TLP5。当 TLP6~7 到达时, 接收端仍然丢弃这些报文, 即便这些报文通过了完整性检查, 因为这些报文的 Sequence 号大于 NEXT_RCV_SEQ。接收端不会为 TLP6~7 发送 NAK DLLP, 因为此时 NAK_SCHEDULED 位有效。

6) 发送端收到 NAK DLLP, 其序号为 4, 此时发送端首先将 TLP3~4 从 Replay Buffer 中清除, 因为 TLP3~4 已经被接收端正确接收, 然后重新发送 TLP5~7。

7) 接收端如果正确接收到 TLP5 时, 发现其 Sequence 号与 NEXT_RCV_SEQ 相等, 将清除 NAK_SCHEDULED 位。

8) 接收端陆续接收到 TLP6~7, 并根据 CRC 的检查结果决定发送 ACK DLLP 或者 NAK DLLP。

在某些情况下, 接收端发送的 NAK DLLP 可能并没有被发送端正确接收, 因此接收端在很长一段时间内都不会得到“发送端重试的”TLP。此时接收端将会择时重发 NAK DLLP, 为此接收端设置了一个 AckNak_LATENCY_TIMER 计数器, 当该计数器溢出时, 接收端将重发 NAK DLLP。该计数器的更新规则如下。

1) 当接收端发送 ACK 或者 NAK DLLP 时, 该计数器重置并开始计数。

2) 接收端为“所有已接收的 TLP”发送了 ACK DLLP 报文时, 或者数据链路层状态为 DL_Inactive 时, 该计数器被重置且保持为 0。

AckNak_LATENCY_TIMER 计数器的阈值是 REPLAY_TIMER 计数器阈值的 $1/3^{\ominus}$ 。当接收端等待的时间超过 AckNak_LATENCY_TIMER 计数器的阈值后, 接收端将重发一个 ACK

$\ominus$ ACKNAK_LATENCY_TIMER 计数器的阈值是 REPLAY_TIMER 阈值的 $1/3$, 可以保证接收端至少可以重发两次 ACK DLLP 给发送端。

DLLP[⊖]。

当发送端发送若干个 TLP 之后，接收端将发送一个 ACK DLLP 作为回应。但是在某些情况下，发送端并没有收到接收端的 ACK DLLP。此时接收端需要在 AckNak_LATENCY_TIMER 计数器溢出时，重新发送 ACK DLLP。从而防止“因为发送端的 REPLAY_TIMER 计数器溢出”，重新进行 PCIe 链路训练，重发更多的 TLP。

7.2.3 数据链路层发送报文的顺序

数据链路层还规定了报文发送的顺序。由上文的描述中，我们可以发现 DLLP 和 TLP 的发送共用一个 PCIe 链路，除此之外物理层的报文 PLP（Physical Layer Packet）也使用同样的链路。因此 PCIe 链路需要合理地安排报文的发送顺序，以避免死锁。其发送顺序如下所示。

1) 正在发送的 TLP 或者 DLLP 具有最高的优先权。PCIe 总线为了保证数据的完整性，不允许打断正在传送的报文。从理论上讲，打断正在传送的报文是可行的，但是硬件需要更大的代价，也需要制定更加复杂的协议保证数据的完整性。

2) PLP 的传送。一般来说，处于协议底层的报文优先权高于处于协议高层的报文，这也是解决死锁的一个有效方法。

3) NAK DLLP。NAK DLLP 需要优先于 TLP 的发送，原理同上。

4) ACK DLLP。ACK DLLP 响应正确接收的报文，在绝大多数处理过程中，错误处理报文优先于正确的响应，这也是一种防止死锁的方法。

5) 重新传送 Replay Buffer 中的 TLP。也是一种发现错误后的恢复手段，因此这种报文的传递优先权高于其他 TLP。因为在错误没有处理完毕之前，其他 TLP 的传递是没有意义的，接收端都将丢弃这些报文。

6) 其他在事务层等待的 TLP。

7) 其他 DLLP，这些 DLLP 包括地址路由，电源管理等报文，这些报文与数据报文的传递无关，是 PCIe 总线规定的一些控制报文，所以优先权最低。

7.3 物理层简介

如图 4-4 所示，物理层在数据链路层和 PCIe 链路之间，其主要作用有三个，一是发送数据链路层的 TLP 和 DLLP；二是发送和接收在物理层产生的报文 PLP；三是从 PCIe 链路接收数据报文并传送到数据链路层。

物理层主要由物理层逻辑模块和物理层电气模块组成，本节主要介绍物理层的逻辑模块，包括 8/10b 编码、链路训练等一些最基础的内容，并通过介绍差分信号的工作原理，简要介绍物理层的电气模块。物理层的电气模块对于深入理解 PCIe 总线规范非常重要，但是许多系统软件工程师因为缺少必要的基础知识，很难理解这部分内容。

本节的内容是第 8 章的基础。如果读者需要深入理解 PCIe 的链路训练，必须掌握本节的全部内容。如果读者对第 8 章内容不感兴趣，可以略过第 8 章和本节。但是 PCIe 总线各个层

[⊖] 注意是重发 ACK DLLP 而不是 NAK DLLP。

次间的联系较为紧密，读者很难在对物理层一无所知的情况下，深入理解 PCIe 总线规范。

物理层的电气模块与差分信号的工作原理密切相关，这部分原理包括一系列与信号完整性相关的课题。而信号完整性本身就是一个专门的话题，其难度与复杂程度较高。信号完整性所追求的目标如下。

- 1) 保证发送的信号可以被接收端正确接收。
- 2) 保证发送的信号不会影响其他信号。
- 3) 保证发送的信号不会损坏接收器件。
- 4) 保证发送的信号不会产生较大的 EMI 电磁噪声。

PCIe 总线的物理层对信号传送进行了一系列约定，以保证信号传递的完整性。而这些约定建立在差分信号传送规则的基础上。如果读者能够深入理解差分信号的工作原理，理解这些约定并不困难。

7.3.1 PCIe 链路的差分信号

PCIe 链路使用差分信号进行数据传递，而差分信号由两个信号组成，这与 PCI 总线使用的单端信号有较大区别。

首先所有信号的传递都需要一个电流回路，而且流入一个节点的电流总和等于流出这个节点的电流总和^①。单端信号使用地平面作为电流回路，而这个地平面并不是稳定的，极易受到干扰，其中最重要的干扰为 SSO（Simultaneous Switching Output）噪声。而减缓 SSO 噪声最有效的方法是为器件的电源提供退耦电容，这个方法也被西方的工程师称为“The rule of thumb”，在电路设计中，该方法极为普及。

即便如此单端信号使用的地平面仍不足以信赖，仍会给单端信号的传递带来不小的干扰。其次单端信号容易收到其他信号的干扰，当单端信号频率较高时，信号在传递过程中衰减较大，而采用差分信号可以有效避免使用单端信号的这些问题。差分信号是由驱动端发送两个等值、相位相反的信号，接收端通过这两个信号的电压差值来判断差分信号是逻辑状态“0”还是“1”。与单端信号相比，差分信号具有许多优势。

1) 抗干扰能力较强。差分信号不受 SSO 噪声的影响，其走线是等长的，且距离较近。当外界存在噪声干扰时，这些干扰同时被耦合到两个信号上，因为接收端只关心这两个信号的差值，这些干扰相减后可以忽略不计。

2) 能够有效抑制信号传递带来的 EMI 干扰。差分信号的极性相反，对外界辐射的电磁场可以相互抵消，因此产生的噪声较小。

3) 逻辑状态定位准确。由于差分信号的开关变化位于两个信号的交点，而不像单端信号使用高低电压两个阈值进行判断，因而受制造工艺、外部环境变化的影响较小，使用差分信号时，接收逻辑较易判断逻辑状态“0”和“1”。

4) 提供的数据带宽较高。由于差分信号受外界环境的影响较小，能够运行在更高的时钟频率上，从而提供的数据带宽较高。

差分信号也有缺点。首先差分信号使用两个信号传递数据，与单端信号相比使用的信号

^① 参见基尔霍夫第一定律。

线较多。但是考虑到单端信号为了保证信号质量,往往使用“两线加一地”^①的方式,因此差分信号使用的信号线数量与单端信号相比,并不是简单乘 2 的关系。其次差分信号的布线与单端信号相比具有较多的约束。差分信号对要求等长且平行走线,而且在实际的 PCB 中,最好做到同层等长,因为不同层间的特性阻抗并不完全相等,而是有一定的误差。

这些约束为差分信号的使用带来了一些困难,但是这些困难并不影响差分信号的大规模应用。目前已知的高速链路均使用差分信号,即将问世的 40 Gb/100 Gb 以太网和 PCIe V3.0 规范也将继续使用差分信号进行数据传递。

差分信号进行传递时依然需要电流回路,而且这个回流路径仍然主要使用地平面,虽然差分信号对彼此也可以作为电流回路,但这并不是主要的回流路径。所有高频信号总是使用电感最小的电流回路,差分信号除了相互间的耦合之外,更多的是对地耦合。

因此差分信号在进行传递时,参考地平面仍然最为重要。如果参考地平面不连续或者不存在参考地平面时,差分信号间的耦合才作为回流通路,但是使用这种方法将极大降低差分信号的质量,而且会增加 EMI 干扰,因此在设计中并不建议使用这种方法。差分信号的传递方法如图 7-10 所示。

如该图所示,差分信号使用两根信号 D+ 和 D- 进行信号传递,其中差分信号可以使用两种方法描述,分别为 (V_1, V_2) 和 (V_{cm}, V_{diff}) 。

假设信号 D+ 的对地参考电压为 V_1 ,而信号 D- 的对地参考电压为 V_2 ,使用 (V_1, V_2) 可以描述一个差分信号,但是这种方法并不常用。因为使用 (V_1, V_2) 这种方法描述差分信号并不直观,接收端更关心这两个信号的差值,即 $V_1 - V_2$ 。

为此我们引入两个参数 (V_{cm}, V_{diff}) ,其中 V_{diff} 参数代表信号 D+ 和信号 D- 的差值电压 (Differential Voltage),而 V_{cm} 参数代表这两个信号的共模电压 (Common Mode Voltage)。这两个参数的计算方法如公式 7-1 所示。

$$\begin{aligned} V_{cm} &= (V_1 + V_2) / 2 \\ V_{diff} &= V_1 - V_2 \end{aligned} \quad (7-1)$$

其中 $V_1 = V_{cm} + V_{diff} / 2$ 而 $V_2 = V_{cm} - V_{diff} / 2$ 。对于差分信号而言, (V_1, V_2) 和 (V_{cm}, V_{diff}) 这两种描述方式是完全等价的。如图 4-1 所示,在 PCIe 链路中,差分信号首先经过一个 AC 耦合电容后,才能到达接收端。因此接收端收到的差分信号,其直流电压已被滤去。因此在实际应用中,接收端并不关心 V_{cm} ,而仅关心 V_{diff} 。但是发送端需要置 V_{cm} 为一个合适的偏置电压,保证信号的传送。在理想情况下,差分信号 D+ 和 D- 相位相反,幅值相等,且 V_{cm} 为一个常量,如图 7-11 所示。

在该图中实线部分为 D+ 信号,而虚线部分为 D- 信号,这两个信号相位完全相反,且为非常理想的正弦波, V_{cm} 为一个恒定的值,而差分电压 V_{diff} 也是一个理想的正弦波,其峰值为 D+ 或者 D- 信号的两倍。

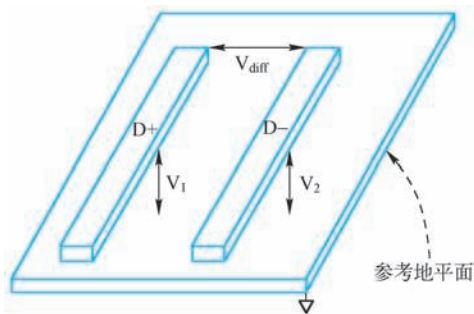


图 7-10 差分信号的传递

① 使用单端数据总线进行长距离传递时,每两根单端信号线之间使用一根地线隔离。

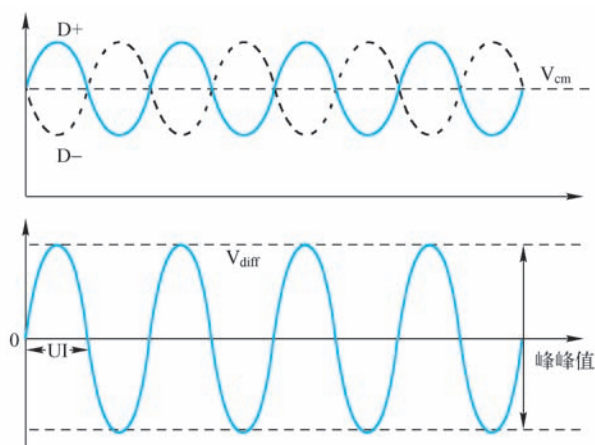


图 7-11 理想的差分信号

然而在差分信号的实际应用中，由于信号 D+ 和 D- 并不会完全对称，相位也不会完全相反，可能会存在一些偏差，如图 7-12 所示。

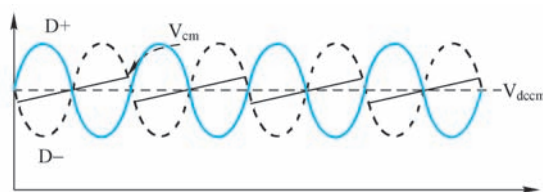


图 7-12 次理想的差分信号

如上图所示，由于 D+ 和 D- 并不完全对称，因此 V_{cm} 并不是一个常数，而是以 V_{dcm} 为中心进行上下波动。 V_{dcm} 为 DC 共模电压（DC Common Mode Voltage），其值为 V_{cm} 在一段时间内的平均直流电压。

与直流共模电压相对应，在对差分信号进行分析时，还使用 AC 共模电压（AC Common Mode Voltage），简称为 V_{acm_rms} ^①，其值为 $RMS[(V_1 + V_2) - V_{dcm}]$ 。其中 RMS（Root Mean Square）用来计算 AC 电压的有效值，对于正弦波而言，RMS 值约等于峰值的 0.707 倍。

在差分信号传递中，使用 UI（Unit Interval）计算单位时间，如图 7-11 所示，UI 的值为一个正弦波的半个周期。在 PCIe V1.x 规范中，使用的时钟频率为 1.25 GHz，因此 UI 在 399.88~400.12 ps 之间；而 PCIe V2.x 规范使用的时钟频率为 2.5 GHz，此时 UI 在 199.94~200.06 ps 之间。PCIe 总线还规定了一系列有关差分信号传递的参数，并分为发送逻辑和接收逻辑^②区别处理，如表 7-3 和表 7-4 所示。

值得注意的是，在本书中发送端与发送逻辑，接收端与接收逻辑是完全不同的概念。如图 4-1 所示，发送端和接收端都包含发送逻辑和接收逻辑，本书为强调发送逻辑和接收逻辑

① 通常交流电压使用有效值表示。

② 如图 4-1 所示，发送端和接收端都有相应的发送逻辑（TX）和接收逻辑（RX）。

辑的概念，使用发送逻辑 TX（Transmitter）和接收逻辑 RX（Receiver）来表示发送端和接收端的发送逻辑和接收逻辑。有的书籍也将发送逻辑和接收逻辑称为发送模块和接收模块。

本节仅列出“发送逻辑 TX”和“接收逻辑 RX”使用的部分参数，对全部参数有兴趣的读者可以参阅 PCIe V2.1 总线规范的表 4-9 和表 4-12。

表 7-3 PCIe 链路“发送逻辑 TX”差分信号的参数

符 号 名	2.5 GT/s	5 GT/s	单 位	描 述
UI	399.88(Min) 400.12(Max)	199.94(Min) 200.06(Max)	ps	时钟误差范围为 ± 300 ppm，因此 UI 存在少许误差
$V_{TX-DIFF-PP}$	0.8(Min) 1.2(Max)	0.8(Min) 1.2(Max)	V	V_{diff} 的峰峰值，等于 $2 V_{D+} - V_{D-} $
$V_{TX-DIFF-PP-LOW}$	0.4(Min) 1.2(Max)	0.4(Min) 1.2(Max)	V	在低电压模式下 V_{diff} 的峰峰值
T_{TX-EYE}	0.75(Min)	0.75(Min)	UI	眼图的宽度
$Z_{TX-DIFF-DC}$	80(Min) 120(Max)	120(Max)	Ω	差分信号的 DC 阻抗
$V_{TX-CM-AC-PP}$	Not specified	100(Max)	mV	AC 共模电压的峰峰值，等于 $\max(V_{D+} + V_{D-})/2 - \min(V_{D+} + V_{D-})/2$
$V_{TX-CM-AC-P}$	20	Not specified	mV	AC 共模电压的有效值，其值等于 $RMS [(V_{D+} + V_{D-})/2 - DC_{AVG}(V_{D+} + V_{D-})/2]^{\text{①}}$
$I_{TX-SHORT}$	90(Max)	90(Max)	mA	发送逻辑 TX 在短路状态下的输出电流
$V_{TX-DC-CM}$	0(Min) 3.6(Max)	0(Min) 3.6(Max)	mV	DC 共模电压
$V_{TX-IDLE-DIFF-AC-p}$	0(Min) 20(Max)	0(Min) 20(Max)	mV	发送逻辑 TX 处于 Electrical Idle 状态时， V_{D+} 和 V_{D-} 的交流电压差值
$V_{TX-IDLE-DIFF-DC}$	Not specified	0(Min) 5(Max)	mV	发送逻辑 TX 处于 Electrical Idle 状态时， V_{D+} 和 V_{D-} 的直流电压差值。
$V_{TX-RCV-DETECT}$	600(Max)	600(Max)	mV	该参数与 Receiver Detect 的过程相关。第 8.1.3 节将详细介绍 Receiver Detect 逻辑
$T_{TX-IDLE-MIN}$	20(Min)	20(Min)	ns	发送逻辑 TX 处于 Electrical Idle 状态时的最短时间
$T_{TX-IDLE-SET-TO-IDLE}$	8(Max)	8(Max)	ns	发送完毕 EIOS 序列后，发送逻辑 TX 进入 Electrical Idle 状态的最短时间
$T_{TX-IDLE-TO-DIFF-DATA}$	8(Max)	8(Max)	ns	发送逻辑 TX 离开 Electrical Idle 状态，到可以发送正常差分信号需要的转换时间
$T_{CROSSLINK}$	1.0(Max)	1.0(Max)	ms	在使用 Crosslink 连接两个 Switch 时使用
$L_{TX-SKEW}$	500+2UI(Max)	500+4UI(Max)	ps	Lane-Lane 间的传送漂移
C_{TX}	75(Min) 200(Max)	75(Min) 200(Max)	nF	发送逻辑 TX 使用的 AC 耦合电容

① $DC_{AVG}(V_{D+} + V_{D-})/2$ 为一段时间内 DC 共模电压的平均值，PCIe 总线要求这段时间至少为 10^6 个 UI。

表 7-4 PCIe 链路“接收逻辑 RX”差分信号的参数

符 号 名	2.5 GT/s	5 GT/s	单 位	描 述
UI	399.88(Min) 400.12(Max)	199.94(Min) 200.06(Max)	ps	与发送逻辑类似
$V_{RX-DIFF-PP-CC}$	0.175(Min) 1.2(Max)	0.120(Min) 1.2(Max)	V	V_{diff} 的峰峰值
$V_{RX-DIFF-PP-DC}$	0.175(Min) 1.2(Max)	0.100(Min) 1.2(Max)	V	
T_{RX-EYE}	0.4(Min)	N/A	UI	眼图的宽度
$T_{RX-MIN-PULSE}$	Not Specified	0.6(Min)	UI	接收端需要的信号最小脉冲间隔
Z_{RX-DC}	40(Min) 60(Max)	40(Min) 160(Max)	Ω	单端信号的 DC 阻抗。该参数的作用是便于发送逻辑 TX 进行 Receiver Detection
$Z_{RX-DIFF-DC}$	80(Min) 120(Max)	Not Specified	Ω	差分信号的 DC 阻抗
$V_{RX-CM-AC-P}$	150(Max)	150(Max)	mV	AC 共模电压
$Z_{RX-HIGH-IMP-DC-POS}$	50k(Min)	50k(Min)	Ω	接收逻辑 RX 的 V_{cc} 没有上电时, 当输入电压大于 0 时 DC 共模输入阻抗
$Z_{RX-HIGH-IMP-DC-NEG}$	1.0k(Min)	1.0k(Min)	Ω	接收逻辑 RX 的 V_{cc} 没有上电时, 当输入电压小于 0 时 DC 共模输入阻抗
$V_{RX-IDLE-DET-DIFFp-p}$	65(Min) 175(Max)	65(Min) 175(Max)	mV	用于 Idle 状态检测的电压阈值。其值等于 $2 V_{RX-D+} - V_{RX-D-} $
$T_{RX-IDLE-DET-DIFFERTIME}$	10(Max)	10(Max)	ms	当 V_{diff} 小于 65mV 时, 发送逻辑 TX 可能已经处于 Electrical Idle 状态。发送逻辑 TX 需要在 10 ms 之内识别出这种“意外”的 Electrical Idle 状态 ^①

① 在正常情况下, 发送模块进入 Electrical Idle 状态之前, 需要发送若干个 EIOS 序列。

以上这些参数将在 PCIe 链路训练和重新训练中使用。在 PCIe 总线中, 和差分信号有关的内容还有许多, 如阻抗的计算、Emphasis (预加重)、De-Emphasis (预去重) 和 PCB 布线等。这些内容并非本书的重点, 对此有兴趣的读者可参考 Howard Johnson 和 Martin Graham 合著的 High-Speed Signal Propagation。

深入理解差分信号的工作原理是理解 PCIe 电气子层的重要基础, 建议对处理器体系结构有兴趣的读者掌握一些基本的信号完整性的理论知识。从 PCIe 体系结构设计角度来看, 信号完整性这部分内容涉及了许多模拟电路的设计与实现知识, 这部分内容是 PCIe 体系结构的精华所在, 但并不是本书侧重的内容。

7.3.2 物理层的组成结构

PCIe 总线的物理层通过 LTSSM 状态机对 PCIe 链路进行配置与管理, 并与数据链路层进行数据交换, 由逻辑子层 (Logical Sub-block) 和电气子层 (Electrical Sub-block) 组成。本节主要讲述逻辑子层。逻辑子层与数据链路层进行数据交换, 由发送逻辑 TX 和接收逻辑 RX 组成, 其结构如图 7-13 所示。

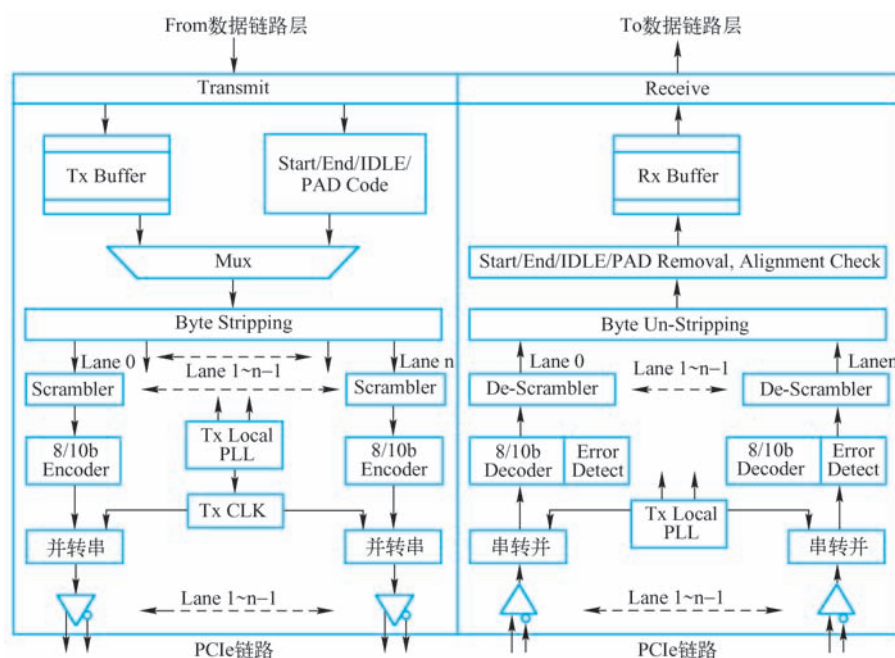


图 7-13 逻辑子层的组成结构

如上图所示，物理层发送报文的过程如下。

- 1) 物理层从数据链路层获得 TLP 或者 DLLP，然后放入 Tx Buffer 中。
- 2) 物理层将这些 TLP 或者 DLLP 加入物理层的前缀（Start Code）和后缀（End Code），后通过多路选择器 Mux，进入 Byte Stripping 部件。物理层也定义了一系列 PLP，这些 PLP 也可以通过 Mux，进入 Byte Stripping 部件。
- 3) PCIe 链路可能由多个 Lane 组成，Byte Stripping 部件可以将数据报文分发到不同的 Lane 中。在 PCIe 链路的不同 Lane 中传递的数据可能存在漂移，即 Skew，Byte Stripping 部件还有一个重要功能即消除这个漂移，即 De-skew。
- 4) 数据进入到各自 Lane 的加扰（Scrambler）部件，“加扰”后进行 8/10b 编码，最后通过并转串逻辑将数据发送到 PCIe 链路中。

物理层的接收过程是发送的逆过程，其步骤如下。

- 1) 物理层从 PCIe 链路的各个 Lane 获得串行数据，并通过 8/10b 解码和 De-Scrambler 部件，发送到“Byte Un-Stripping”部件。
- 2) “Byte Un-Stripping”部件将来自不同 Lane 的数据合并，进行 De-skew 操作，然后取出物理层的前后缀并进行边界检查后，将数据放入 Rx Buffer 中。
- 3) 物理层将在 Rx Buffer 中的数据传递到数据链路层。

物理层的数据在通过 Byte Un-Stripping/Stripping 部件时，需要注意大小端模式的转换。而 Scrambler 和 De-Scrambler 部件的主要作用是对数据流进行“加扰”和“解扰”操作。在串行链路上进行数据传递时，如果在字符流中存在某些规律，这些“规律”将会叠加，并产生较大的 EMI（Electromagnetic interference）噪声。

Scrambler 部件的主要作用就是通过“加扰”的方法削减 EMI 噪声，所谓加扰是指将源

数据流与一个随机序列进行异或操作后，再发送出去。此时被发送出的数据流也基本是伪随机的，从而降低了发送数据时产生的 EMI 噪声。

PCIe 总线通过一个 16 位线性反馈移位寄存器（Linear Feedback Shift Register, LFSR），产生伪随机序列，该移位寄存器的表达式如公式 7-2 所示。

$$G(X) = X^{16} + X^5 + X^4 + X^3 + 1 \quad (7-2)$$

该公式是一个本原多项式，使用该本原多项式可以产生一个周期为 $2^{16}-1$ （这个周期是 16 位移位寄存器能够产生的最大周期）的伪随机序列。所谓本原多项式是“具有最大周期”的不可约多项式。对应的，由本原多项式作为生成多项式所产生的 LFSR 序列为最大周期序列。这些序列一般被称为 m-序列，在 m-序列中“0”和“1”所占的比例相对均衡，但是 1 的个数比 0 的个数多 1，因为全 0 不能作为初始值，也不可能是中间状态。

来自 Byte Stripping 部件的字符流与这个伪随机序列中的字符流进行异或操作，从而生成一个相对较为随机的字符流，从而降低了数据流的 EMI 噪声。

De-Scrambler 部件的主要作用是进行解扰。值得注意的是，在 PCIe 链路的两端，加扰和解扰使用的编解码公式相同，而且完全同步，即 LFSR 使用相同的初始值，在 PCIe 链路的两端，该初始值为 0xFFFF。PCIe 链路两端设备每次加解扰一个 8b 数据后，LFSR 进行 8 次移位操作。在 PCIe 总线中，数据在发送时，首先经过“加扰”操作，然后进入 8/10b 编码模块；而接收数据时，首先经过 8/10b 解码模块，然后进行“解扰”操作。

7.3.3 8/10b 编码与解码

IBM 于 1983 年提出 8/10b 编码方法，这个编码方法也是 IBM 的专利。目前这个专利已经过期，以太网、ATM、Infiniband 和 FC（Fiber Channel）在物理链路的数据传送中也使用了 8/10b 编码技术。8/10b 编码是高速串行总线常用的编码方式。

该编码将 8 位编码转化为 10 位，以平衡数据流中 0 与 1 的数量。使用这种方法可以保证数据流中 1 和 0 的数量相等，即保证直流平衡（DC Balance）。如果在一个高速串行数据流中有较多连续的“1”时，会将 AC 耦合电容充满，从而影响这些电容的正常工作，在 PCIe 链路上，AC 耦合电容的位置如图 4-1 所示。

PCIe V1.x 和 2.x 规范使用了 8/10b 编码方式，而 V3.0 规范将使用 128/130b 编码方式。128/130b 编码方式与 8/10b 编码方式原理较为类似，使用 128/130b 编码可以进一步提高 PCIe 总线的利用率，但是需要更多的硬件资源。本节仅介绍 8/10b 编解码方式。在 PCIe 总线中，编码与解码的过程如图 7-14 所示。

8/10b 编码的基本原理是将一个连续的 8 位数据流分为两组，其中一组由 3 位（FGH）组成，而另一组由 5 位（ABCDE）组成。8/10b 编码将这两组数据流分别编码成一组 4 位（fghj）和一组 6 位（abcdei）的数据流。而解码过程是编码的逆过程，将一组 4 位和一组 6 位的数据流还原成为一组 3 位和一组 5 位的数据流。

PCIe 设备采用这种编码方式可以保证数据流中出现的 0 和 1 的数量基本保持一致，同时保证在通过 PCIe 物理链路的数据流中，连续的“1”和“0”不会超过 5 个。虽然采用 8/10b 编码将降低总线的使用效率，但是能够保证高速串行信号的传送完整性。

如图 7-13 所示，物理层可以对两类字符进行 8/10b 编码，一类是数据字符，即从数据链路层获得的 TLP 和 DLLP；一类是物理层使用的控制字符，如 Start/End/Idle

Code 和一些物理层中使用的 PLP。为此 PCIe 总线在进行 8/10b 编解码需要区分数据和控制字符。

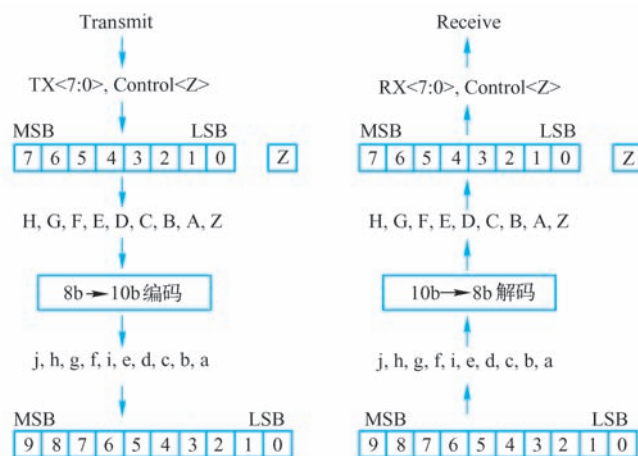


图 7-14 8/10b 编解码过程

数据字符与控制字符使用的 8/10b 编码不同，PCIe 总线分别使用 D_{xx.y} 和 K_{xx.y} 表示数据字符 (D) 和控制字符 (K)，其中 xx 记录字符的低 5 位 ABCDE，而 y 记录字符的高 3 位 FGH。

值得注意的是，PCIe 总线使用 8/10b 编码可以保证每十位中，最多有 6 个 1 或者 6 个 0，而不是传统 8/10b 编码中要求的“不超过 5 个连续的 0 或者 1”。使用这种方法基本上可以保证数据流的 DC 平衡。但是使用这种编码方式无法保证在某些特殊情况中，连续发送的数据流中都含有“6 个 1，4 个 0”或者“6 个 0，4 个 1”。随着时间的累积，这些数据流依然会在链路中造成严重的 DC 失衡。

为此 PCIe 总线使用 CRD (Current Running Disparity) 技术进一步保证 PCIe 链路的 DC 平衡。PCIe 总线在进行 8/10b 编码时，每一个 D_{xx.y} 和 K_{xx.y} 对应两个 10b 的编码，分别是 CRD+ 和 CRD-。对于多数编码，CRD- 和 CRD+ 中含有的“0”和“1”的个数相同，如 D1.0、D2.0 等数据字符。但是在有些 CRD+ 编码中，“1”的个数小于“0”的个数；而在 CRD- 编码中“0”的个数大于“1”的个数，如 D1.1 和 D2.1 的编码。

在 CRD+ 编码中，“1”的个数小于或者等于“0”的个数；在 CRD- 编码中，“0”的个数小于或者等于“1”的个数。值得注意的是，CRD+ 和 CRD- 编码并不是直接取反的关系，当 CRD 编码的“0”的个数与“1”的个数相同时，CRD+ 与 CRD- 的编码有时是相同的，如 D3.5 的编码。

下文以一个数据发送的实例说明 CRD+、CRD- 编码的使用。在 PCIe 链路的发送端中，存在一个 CRD 状态位，其初始值可以为“正”或者“负”。随着通过 PCIe 链路的数据流增多，累积的“1”和“0”的个数可能并不平衡。当所有通过 PCIe 链路的字符流中，“1”的个数大于“0”的个数时，CRD 状态为正；当所有通过 PCIe 链路的字符流中，“0”的个数大于“1”的个数时，CRD 状态为负；当所有通过 PCIe 链路的字符“0”的个数等于“1”的个数时，CRD 状态保持不变。

当 CRD 状态为正时，PCIe 链路进行 8/10b 编码时使用 CRD+，否则使用 CRD-以维持 PCIe 链路的 DC 均衡。在 PCIe 总线中，数据字符使用的 8/10b 编码的格式如表 7-5 所示。

表 7-5 数据字符的 8/10b 编码

数据字符	Data Byte	HGF EDCBA	CRD-abcd e fghi	CRD+abcd e fghi
D0. 0	0x00	000 00000	100111 0100	011000 1011
D1. 0	0x01	000 00001	011101 0100	100010 1011
D2. 0	0x02	000 00010	101101 0100	010010 1011
D3. 0	0x03	000 00011	110001 1011	110001 0100
D4. 0	0x04	000 00100	110101 0100	001010 1011
D5. 0	0x05	000 00101	101001 1011	101001 0100
D6. 0	0x06	000 00110	011001 1011	011001 0100
D7. 0	0x07	000 00111	111000 1011	000111 0100
...				
D1. 1	0x21	001 00001	011101 1001	100010 1001
D2. 1	0x22	001 00010	101101 1001	010010 1001
D3. 1	0x23	001 00011	110001 1001	110001 1001
...				
D3. 5	0xA3	101 00011	110001 1010	110001 1010
...				
D23. 7	0xF7	111 10111	111010 0001	000101 1110
D24. 7	0xF8	111 11000	110011 0001	001100 1110
D25. 7	0xF9	111 11001	100110 1110	100110 0001
D26. 7	0xFA	111 11010	010110 1110	010110 0001
D27. 7	0xFB	111 11011	110110 0001	001001 1110
D28. 7	0xFC	111 11100	001110 1110	001110 0001
D29. 7	0xFD	111 11101	101110 0001	010001 1110
D30. 7	0xFE	111 11110	011110 0001	100001 1110
D31. 7	0xFF	111 11111	101011 0001	010100 1110

PCIe 总线还定义了一系列控制字符，这些字符从“Data Byte”的角度来看和数据字符完全相同，但是使用的 CRD+和 CRD-编码和数据字符不同。如数据字符 D30. 7 和 K30. 7 所对应的“Data Byte”都为 0xFE (111 11110)，但是 CRD-编码分别为 011110 0001 和 011110 1000，而 CRD+编码为 100001 1110 和 100001 0111。

控制字符使用的 8/10b 编码的格式如表 7-6 所示。PCIe 总线使用这些字符编码作为控制命令，和数据进行区别。

表 7-6 控制字符的 8/10b 编码

数 据 字 符	Data Byte	HGF EDCBA	CRD-abcdei fghi	CRD+abcdei fghi
K28. 0	0x1C	000 11100	001111 0100	110000 1011
K28. 1	0x3C	001 11100	001111 1001	110000 0110
K28. 2	0x5C	010 11100	001111 0101	110000 1010
K28. 3	0x7C	011 11100	001111 0011	110000 1100
K28. 4	0x9C	100 11100	001111 0010	110000 1101
K28. 5	0xBC	101 11100	001111 1010	110000 0101
K28. 6	0xDC	110 11100	001111 0110	110000 1001
K28. 7	0xFC	111 11100	001111 1000	110000 0111
K23. 7	0xF7	111 10111	111010 1000	000101 0111
K27. 7	0xFB	111 11011	110110 1000	001001 0111
K29. 7	0xFD	111 11101	101110 1000	010001 0111
K30. 7	0xFE	111 11110	011110 1000	100001 0111

这些控制字符在 PCIe 总线中的定义如表 7-7 所示。

表 7-7 控制字符的说明

数 据 字 符	缩 写	符 号 名	说 明
K28. 0	SKP	Skip	用于补偿 PCIe 链路不同 Lane 的延时，PCIe 总线的物理层收到该控制序列时，LFSR 不进行移位操作
K28. 1	FTS	FTS (Fast Training Sequence)	在链路训练的 FTS 序列中使用
K28. 2	SDP	Start DLLP	DLLP 的起始标记
K28. 3	IDL	Idle	在 EIOS (Electrical Idle Ordered Set) 序列中使用
K28. 4			保留
K28. 5	COM	Comma	复位 PCIe 链路的 LFSR 为初始值
K28. 6			保留
K28. 7	EIE	Electrical Idle Exit	在 EIEOS (Electrical Idle Exit Sequence) 序列中使用
K23. 7	PAD	Pad	填充字符
K27. 7	STP	Start TLP	TLP 的起始标志
K29. 7	END	End	TLP 和 DLLP 的结束标志
K30. 7	EDB	EnD Bad	无效 TLP 的结束标志

下文以物理层发送一个 TLP 的实例说明，PCIe 链路如何使用这些编码。一个 TLP 在通过物理层时，首先要加入物理层的前后缀，分别为 STP 和 END。加入这些前后缀后的 TLP 报文格式如图 7-15 所示。

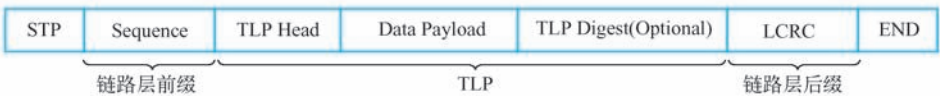


图 7-15 物理层 TLP 的报文格式

TLP 在通过物理层时首先在其前后加入 STP 和 END 控制字符，这两个控制字符分别为 K27.7 和 K29.7（如表 7-7 所示），它们通过物理层时，不需要进行加接扰操作。数据链路层前缀、TLP 和数据链路层后缀都属于数据字符，这些字符在通过物理层时需要进行加接扰操作，之后从表 7-5 中获得字符流，并由物理层发向 PCIe 链路。

值得注意的是，控制字符和数据字符需要根据物理层 CRD 状态，决定是使用 CRD+ 还是 CRD- 编码。PCIe 链路的两端在进行加解扰操作时，需要保证其使用的 LFSR 寄存器同步。LFSR 寄存器的同步由控制字符 COM 决定，在初始复位时 LFSR 寄存器的初始值为 0xFFFF，当收到控制字符 COM 后，物理层将 LFSR 寄存器的初始值置为 0xFFFF，此外物理层收到控制字符 SKP 后，并不会对 LFSR 寄存器进行移位操作。

7.4 小结

本章重点介绍了数据链路层的状态，以及 ACK/NAK 协议，并简要介绍了 PCIe 总线的物理层。其中 PCIe 总线的物理层非常重要，深入理解物理层是深入理解 PCIe 体系结构的要点。在第 8 章讲述的内容以此为基础。